
ERP Pro

Complete System Documentation

Version 1.0 • August 2026
Proprietary — Internal Use Only

Table of Contents

- 1. Introduction
- 2. Technical Architecture
- 3. Database Design
- 4. Modules Overview
- 5. Role-Based Access Control (RBAC)
- 6. Security Model
- 7. Competitive Analysis
- 8. Complete API Reference

1. Introduction

ERP Pro is a comprehensive, multi-company, multi-branch Enterprise Resource Planning system built with modern web technologies. It provides a unified platform for managing the complete business lifecycle: from sales and purchasing through inventory management, accounting, human resources, point of sale, project management, and manufacturing.

The system is designed as a SaaS-ready platform that can be self-hosted or deployed in the cloud. It features a professional dark-mode UI with full bilingual support (Arabic and English), granular role-based access control with 63 permission codes, and a modular architecture where companies only enable the modules they need.

Key System Statistics

Metric	Value	Description
API Endpoints	~120	RESTful endpoints across 18 routers
Database Tables	40+	PostgreSQL tables with full referential integrity
Permission Codes	63	Granular view/manage/delete per module
Licensed Modules	8	Sales, Purchases, Inventory, Accounting, HR, POS, Projects, Manufacturing
Test Coverage	110 + 24	110 unit/integration tests + 24 E2E browser tests
Languages	2	English and Arabic with RTL support
ORM Models	40+	SQLAlchemy 2.0 declarative models

Design Principles

Multi-Tenant First

Every business table has a `company_id` foreign key. All queries are automatically filtered by the authenticated user's current company. Users can belong to multiple companies with different roles in each.

Module Licensing

Each company has an `enabled_modules` list. The `require_module` dependency checks this before granting access to module-specific endpoints. Superusers bypass this check.

Fail-Open Design

Redis outages do not block logins (rate limiting degrades gracefully). Email failures are logged but never crash the application. The system is designed to remain operational under partial infrastructure failures.

Convention Over Configuration

Auto-generated sequential codes for invoices, work orders, projects. Weighted-average costing calculated automatically on purchase confirmation. Stock adjustments posted automatically on stock take completion.

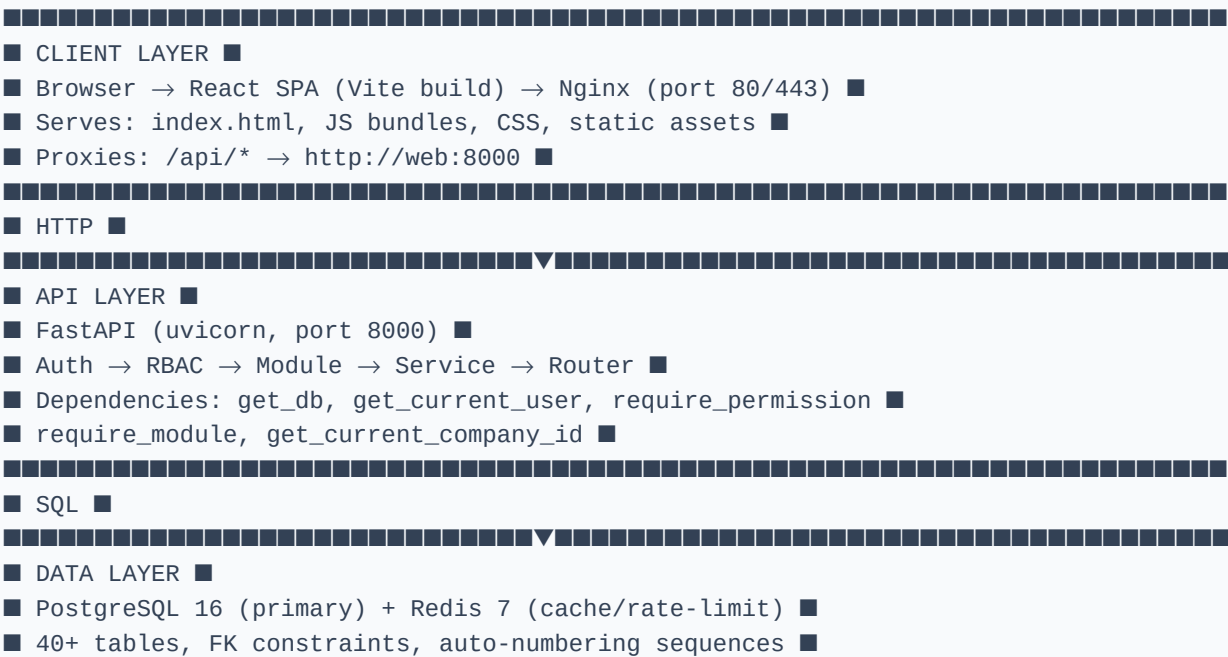
2. Technical Architecture

Technology Stack

Layer	Technology	Version	Purpose
Backend Framework	FastAPI	0.115+	Async REST API with automatic OpenAPI docs
ORM	SQLAlchemy	2.0	Declarative models with type hints
Migrations	Alembic	1.14+	Zero-downtime schema migrations
Database	PostgreSQL	16	Primary data store with full ACID compliance
Cache / Queue	Redis	7	Rate limiting, session caching, pub/sub
Frontend Framework	React	19	Component-based SPA with hooks
Build Tool	Vite	8	Fast HMR and optimized production builds
HTTP Client	Axios	1.7+	API communication with interceptors
Routing	React Router	7	Client-side routing with lazy loading
Containerization	Docker	24+	Reproducible builds and deployments
Reverse Proxy	Nginx	1.27	SPA serving + API proxy in production
Testing	pytest	9+	Unit, integration, and E2E testing
PDF Generation	ReportLab	4+	Professional documentation generation

System Architecture Diagram

The system follows a classic three-tier architecture with clear separation of concerns:



Request Flow

1. Client sends HTTP request to Nginx (port 80/443)
2. Nginx proxies /api/* to FastAPI backend (port 8000)
3. FastAPI extracts bearer token from Authorization header
4. get_current_user dependency validates token → returns User
5. get_current_company_id dependency reads session → returns company_id
6. require_permission dependency checks user's roles → grants or returns 403
7. require_module dependency checks company_settings.enabled_modules → 403 if not licensed
8. Route handler calls service layer with validated data
9. Service layer executes business logic via SQLAlchemy ORM
10. Response serialized via Pydantic schema → returned as JSON

Deployment Architecture

The system is deployed as a Docker Compose stack with 4 services:

Service	Image	Port	Role
db	postgres:16-alpine	5432 (internal)	Primary database
redis	redis:7-alpine	6379 (internal)	Rate limiting + cache
web	custom (Python 3.12)	8000 (internal)	FastAPI backend
frontend	custom (Nginx 1.27)	80 → 9009 (host)	SPA + API proxy

3. Database Design

The database contains 40+ tables organized into functional domains. Every business table has a `company_id` foreign key for multi-tenant isolation. All foreign keys use `ON DELETE CASCADE` except `AuthSession.current_company_id` which uses `SET NULL` (to preserve session history).

Entity Relationship Overview

Domain	Tables	Key Relationships
Auth	users, auth_sessions	auth_sessions.user_id → users.id
Companies	companies, branches, company_settings	branches.company_id → companies.id
RBAC	roles, permissions, role_permissions, user_roles	user_roles links users to companies with roles
Partners	partners	partners.company_id → companies.id
Items	items, item_categories, units, unit_conversions	items → categories, items → units
Inventory	warehouses, warehouse_stock, inventory_movements	warehouse_stock → warehouses, items
Sales	sales_invoices, sales_invoice_lines	lines.invoice_id → invoices.id (CASCADE)
Purchases	purchase_invoices, purchase_invoice_lines	lines.invoice_id → invoices.id (CASCADE)
Payments	payments	payments.partner_id → partners.id
Accounting	accounts, journal_entries, journal_lines	journal_lines → entries, accounts
Currencies	currencies, currency_rates	rates.company_id + currency_code → currencies
HR	departments, employees, attendance_records, payroll_runs, payroll_lines, leave_requests	employees → departments
Projects	projects, project_cost_lines	cost_lines.project_id → projects.id
Manufacturing	boms, bom_lines, work_orders, work_order_*	BOMs define components, Work Orders consume them
POS	pos_sessions, pos_orders, pos_order_lines	orders → sessions, lines → orders
Infrastructure	numbering_sequences	Per-company sequential code generation

Multi-Tenant Data Isolation

Every database query that touches business data is automatically filtered by `company_id`. This is enforced at the API layer through the `get_current_company_id()` FastAPI dependency, which reads the `current_company_id` from the user's auth session. The dependency returns HTTP 409 if no company is selected.

The ORM models do not include `company_id` in query filters at the model level — isolation is purely at the API/service layer. This means direct database access (e.g., via `psql`) bypasses the isolation. For production deployments, database-level row security policies can be added as an additional layer of defense.

4. Modules Overview

4.1 Sales Module

Purpose: Manage the complete sales lifecycle: create draft invoices, add line items with products and pricing, confirm invoices to trigger stock deduction and cost calculation, and delete draft invoices.

Key Models: SalesInvoice (id, company_id, partner_id, number, date, currency_code, fx_rate, total_amount, total_amount_base, is_confirmed), SalesInvoiceLine (id, invoice_id, item_id, description, quantity, unit_price, line_total, cost_price, total_cost)

Auto-Numbering: Invoices are numbered per-company using sequential numbering (SI-0001, SI-0002, ...). The numbering uses SELECT ... FOR UPDATE to prevent race conditions.

Confirmation Logic: On confirm: (1) validate all lines have items, (2) calculate total_amount and total_amount_base using fx_rate, (3) for each line: deduct quantity from warehouse_stock, record inventory_movement, calculate cost_price from average_cost, update line.total_cost.

Permissions: sales.view (read), sales.manage (create/update/confirm), sales.delete (delete drafts only)

Endpoints: GET /sales-invoices, POST /sales-invoices, GET /sales-invoices/{id}, PATCH /sales-invoices/{id}, POST /sales-invoices/{id}/confirm, DELETE /sales-invoices/{id}

4.2 Purchases Module

Purpose: Manage the complete purchase lifecycle: create draft invoices, add line items with supplier pricing, confirm invoices to trigger stock-in and weighted-average cost recalculation.

Key Models: PurchaseInvoice, PurchaseInvoiceLine — mirrors the Sales models with supplier-facing semantics.

Confirmation Logic: On confirm: (1) validate lines, (2) calculate totals with FX, (3) for each line: add quantity to warehouse_stock, recalculate $\text{weighted_average_cost} = (\text{old_qty} \times \text{old_avg} + \text{new_qty} \times \text{unit_cost}) / (\text{old_qty} + \text{new_qty})$, record inventory_movement.

Cost Method: Weighted Average Cost (WAC) is the default and only supported method. The average_cost field on warehouse_stock is updated atomically on each purchase confirmation.

Permissions: purchases.view, purchases.manage, purchases.delete

4.3 Inventory Module

Purpose: Track stock levels across multiple warehouses, record all inventory movements, and support physical stock taking with adjustment posting.

Key Models: Warehouse, WarehouseStock (per item per warehouse), InventoryMovement (every stock change is recorded here), StockTake, StockTakeLine.

Stock Balances: warehouse_stock tracks current quantity and average_cost per item per warehouse. The $\text{total_value} = \text{quantity} \times \text{average_cost}$.

Movements: Every stock change creates an inventory_movement record with: item_id, warehouse_from_id, warehouse_to_id, quantity, movement_type (purchase_in, sale_out, manufacturing_in, manufacturing_out, transfer, adjustment), unit_cost, total_cost.

Stock Takes: Physical counting workflow: (1) create stock take for a warehouse, (2) add items with book_qty (auto-fetched) and counted_qty, (3) system calculates diff_qty = counted - book, (4) on post: create adjustment movements and update warehouse_stock.

Permissions: warehouses.view/manage/delete, stock.view, movements.view, stock_takes.view/manage

4.4 Accounting Module

Purpose: Full double-entry bookkeeping with chart of accounts, journal entries, and three financial reports.

Chart of Accounts: Pre-seeded with 13 accounts covering: Cash, Accounts Receivable, Inventory, Fixed Assets, Accounts Payable, Sales Tax Payable, Salaries Payable, Owner's Equity, Sales Revenue, COGS, Operating Expenses, FX Gain, FX Loss. Companies can add custom accounts.

Journal Entries: Double-entry: each entry has multiple journal_lines, each with debit or credit amount linked to an account. The system validates that total debits = total credits before posting.

Reports: Trial Balance (all accounts with debit/credit totals), Income Statement (revenue - COGS - expenses = net income), Balance Sheet (assets = liabilities + equity). All reports are generated dynamically from journal_lines.

Permissions: accounting.view, accounting.manage, accounting.reports

4.5 HR Module

Purpose: Manage employees, departments, attendance tracking, payroll processing, and leave requests.

Departments: Organizational units. Each employee belongs to one department.

Employees: Core employee record with: employee_number (auto-generated), name, position, department, hire_date, basic_salary, is_active.

Attendance: Daily attendance recording per employee: present, absent, half_day, leave statuses. Used for payroll calculation.

Payroll: Period-based payroll runs. On run: for each active employee, calculate basic_salary - deductions = net_salary. Records are created as PayrollLine records under a PayrollRun.

Leave Requests: Workflow: employee submits request (type: annual/sick/unpaid, start_date, end_date, reason) → manager approves or rejects → status updated.

Permissions: hr.view, hr.manage, hr.payroll

4.6 POS Module

Purpose: Session-based point of sale for retail operations with cash management.

Sessions: Open session (record opening_cash) → process orders → close session (enter closing_cash, system calculates expected_cash and variance).

Orders: Each order is linked to a session, contains order_lines (item, quantity, unit_price), and generates a sales invoice on completion.

Permissions: pos.view, pos.manage

4.7 Projects Module

Purpose: Track project profitability with cost allocation across materials, labor, and overhead.

Cost Lines: Each project has multiple cost_lines categorized as material, labor, or overhead. Total cost is aggregated automatically.

Workflow: Create project (with contract_value) → add cost lines throughout project life → mark complete → system calculates margin = contract_value - total_cost.

Permissions: projects.view, projects.manage

4.8 Manufacturing Module

Purpose: Define bills of materials and execute work orders with full cost tracking.

BOMs: Bill of Materials: defines the output product, output quantity, and required component items with quantities.

Work Orders: Production execution: select BOM or product, planned quantity, link to warehouse. On finish: consume component materials from warehouse, add labor and overhead costs, add finished goods to warehouse.

Cost Allocation: Work orders track: total_material_cost (from consumption), total_labor_cost (from labor lines), total_overhead_cost (from overhead lines), total_cost (sum of all three).

Permissions: manufacturing.view, manufacturing.manage

5. Role-Based Access Control (RBAC)

The RBAC system provides granular, per-company access control with 63 permission codes organized across all modules. Permissions follow the format: module.action.

Permission Tiers

Tier	Pattern	Example	Access Level
View	.view	sales.view	Read-only access to module data
Manage	.manage	sales.manage	Create, update, confirm operations
Delete	.delete	sales.delete	Permanent deletion (separate from manage)
Reports	.reports	accounting.reports	Access to module-specific reports
Special	companies.delete_data	—	Danger zone: clear all company data

Permission Resolution

When a user makes an API request, the `require_permission('code')` dependency performs the following resolution:

1. Extract `user_id` from the authenticated session
2. Extract `company_id` from the session's `current_company_id`
3. Query `user_roles` WHERE `user_id = ?` AND `company_id = ?`
4. For each role, query `role_permissions` to get all `permission_ids`
5. Union all permission codes across all roles
6. Check if the required permission code exists in the union
7. If found: proceed. If not: return HTTP 403 Forbidden

Role Management

Each company starts with a built-in 'Admin' role that automatically receives ALL 63 permissions. This role cannot be deleted or have its permissions reduced. Companies can create custom roles and assign specific permissions via the RBAC API or the Roles management page in the UI.

Complete Permission List

Permission Codes	Description
partners.view / .manage / .delete	Customer and supplier management
categories.view / .manage / .delete	Item category management
units.view / .manage / .delete	Unit of measure management

Permission Codes	Description
unit_conversions.view / .manage / .delete	Unit conversion rules
items.view / .manage / .delete	Product and service management
warehouses.view / .manage / .delete	Warehouse management
stock.view	View stock balances
movements.view	View inventory movements
stock_takes.view / .manage	Physical stock counting
sales.view / .manage / .delete	Sales invoice operations
purchases.view / .manage / .delete	Purchase invoice operations
payments.view / .manage	Payment recording
currencies.view / .manage / .delete	Currency management
currency_rates.view / .manage / .delete	Exchange rate management
accounting.view / .manage / .reports	Chart of accounts, journals, reports
hr.view / .manage / .payroll	HR and payroll operations
projects.view / .manage	Project management
manufacturing.view / .manage	BOM and work order management
pos.view / .manage	Point of sale operations
companies.view / .manage	Company settings
companies.delete_data	Danger zone: clear all operational data
roles.view / .manage	Role and permission management
users.view / .manage	Company user management

6. Security Model

6.1 Authentication

The system uses opaque bearer tokens (not JWT). On login, a cryptographically random token is generated using `secrets.token_urlsafe(32)`. Only the SHA-256 hash of the token is stored in the `auth_sessions` table. The raw token is returned to the client once and never stored server-side. Tokens expire after `ACCESS_TOKEN_EXPIRE_MINUTES` (default: 1440 = 24 hours). Sessions can be revoked via the logout endpoint.

6.2 Rate Limiting

Redis-backed rate limiting protects the login endpoint. Per (IP + email): 5 max attempts in a 15-minute window with 5-minute lockout. Per IP (distributed spray protection): 20 max attempts in 15 minutes. The system fails open — if Redis is unreachable, login proceeds without rate limiting. Counters are cleared on successful login.

6.3 Password Policy

Minimum 8 characters. Strength scoring based on: length (2+ chars=1pt, 8+=1pt), lowercase (1pt), uppercase (1pt), digit (1pt), symbol (1pt). Minimum score: 3/5. Passwords are hashed using `bcrypt` with automatic salting.

6.4 CORS Protection

Configurable via `CORS_ORIGINS` environment variable. Defaults to same-origin (empty string). In production with a single origin (Nginx proxy), no CORS configuration is needed.

6.5 Security Headers

The Nginx configuration serves the following security headers: `Content-Security-Policy` (default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'), `X-Frame-Options`: DENY, `X-Content-Type-Options`: nosniff, `X-XSS-Protection`: 1; mode=block, `Referrer-Policy`: strict-origin, `Permissions-Policy`: camera=(), microphone=(), geolocation=()

6.6 Data Isolation

Every business query is filtered by `company_id` at the API layer. Users can only access data for companies they have a role in. The `get_current_company_id` dependency enforces this on every endpoint that touches business data. Direct database access bypasses this — for production, consider PostgreSQL Row-Level Security policies.

7. Competitive Analysis

ERP Pro is positioned in the market as a self-hosted, bilingual ERP system targeting small and medium businesses. The following comparison highlights key differentiators:

Feature	ERP Pro	Odoo	ERPNext	Rushd	Zoho
Hosting	Self-hosted	Cloud/Hosted	Cloud/Hosted	Cloud only	Cloud only
Price	FREE (self-host)	\$20-40/user/mo	\$50/tenant/mo	2-5K EGP/mo	\$15-60/user/mo
Source Code	Full access	Enterprise only	Open source	Proprietary	Proprietary
Languages	EN + AR (RTL)	Multi-lang	Multi-lang	AR only	EN mainly
RBAC Granularity	63 codes (VMD)	Group-based	Role-based	Basic roles	Role-based
Module Licensing	Per-company	Per-user	All-included	Tiered	Per-user
POS Built-in	Yes (session)	Yes (complex)	Yes (basic)	Basic	No
Manufacturing	Yes (BOM+WO)	Yes (advanced)	Yes (basic)	No	No
Deployment	Docker one-cmd	Complex	Docker/bench	Cloud only	Cloud only
API Docs	Auto (Swagger)	Auto (Swagger)	Auto (Swagger)	Limited	REST only
Tests	110+ unit + 24 E2E	Extensive	Extensive	Unknown	Unknown

Key Differentiators

- ✓ True bilingual AR/EN with proper RTL layout — not an afterthought or plugin
- ✓ Self-hosted with full source code — complete data sovereignty, no vendor lock-in
- ✓ Granular view/manage/delete permissions per module — not just role-level
- ✓ Module licensing — companies only pay for and see the modules they use
- ✓ Docker one-command deployment — identical behavior on laptop and production server
- ✓ Weighted-average costing built-in — recalculated automatically on each purchase
- ✓ Auto-numbering with row-level locking — prevents duplicate invoice numbers under concurrency
- ✓ Platform admin panel — manage all tenants from a single superuser dashboard
- ✓ 110 automated tests + 24 E2E browser tests — verified reliability
- ✓ Low-stock email alerts with configurable thresholds per item and per company

8. Complete API Reference

All endpoints are prefixed with `/api/v1`. Authentication is via Bearer token in the Authorization header. The complete interactive API documentation is available at `/api/v1/docs` (Swagger UI) when the server is running.

Module	Method	Endpoint	Permission	Description
Auth	POST	<code>/auth/login</code>	None	Rate-limited login
Auth	POST	<code>/auth/logout</code>	Bearer	Revoke session
Auth	POST	<code>/auth/select-company</code>	Bearer	Set active company/branch
Auth	GET	<code>/auth/me</code>	Bearer	User info + permissions
Companies	GET	<code>/companies</code>	companies.view	List user's companies
Companies	GET	<code>/companies/current</code>	companies.view	Current company details
Companies	GET	<code>/companies/settings</code>	companies.view	Company settings
Companies	PATCH	<code>/companies/settings</code>	companies.manage	Update settings
Companies	POST	<code>/companies/{id}/branches</code>	companies.manage	Create branch
Companies	POST	<code>/companies/current/danger/clear-data</code>	companies.delete_data	Clear operational data
RBAC	GET	<code>/permissions</code>	roles.view	List all permission codes
RBAC	GET	<code>/roles</code>	roles.view	List company roles
RBAC	POST	<code>/roles</code>	roles.manage	Create role
RBAC	PATCH	<code>/roles/{id}/permissions</code>	roles.manage	Update role permissions
RBAC	DELETE	<code>/roles/{id}</code>	roles.manage	Delete role (must be empty)
RBAC	GET	<code>/company-users</code>	users.view	List company users
RBAC	POST	<code>/company-users</code>	users.manage	Create company user
RBAC	PATCH	<code>/company-users/{id}/roles</code>	users.manage	Update user roles
RBAC	PATCH	<code>/company-users/{id}/status</code>	users.manage	Activate/deactivate
Platform	GET	<code>/platform/modules</code>	superuser	List sellable modules
Platform	GET	<code>/platform/companies</code>	superuser	List all tenants
Platform	POST	<code>/platform/companies</code>	superuser	Create tenant
Platform	GET	<code>/platform/companies/{id}</code>	superuser	Tenant details
Platform	PATCH	<code>/platform/companies/{id}</code>	superuser	Update tenant

Module	Method	Endpoint	Permission	Description
Platform	DELETE	/platform/companies/{id}	superuser	Delete tenant (cascades)
Partners	GET	/partners	partners.view	List partners
Partners	POST	/partners	partners.manage	Create partner
Partners	GET	/partners/{id}	partners.view	Partner details
Partners	PATCH	/partners/{id}	partners.manage	Update partner
Partners	DELETE	/partners/{id}	partners.delete	Delete partner
Items	GET	/items	items.view	List items
Items	POST	/items	items.manage	Create item
Items	PATCH	/items/{id}	items.manage	Update item
Items	DELETE	/items/{id}	items.delete	Delete item
Categories	GET	/item-categories	categories.view	List categories
Categories	POST	/item-categories	categories.manage	Create category
Units	GET	/units	units.view	List units
Units	POST	/units	units.manage	Create unit
Warehouses	GET	/warehouses	warehouses.view	List warehouses
Warehouses	POST	/warehouses	warehouses.manage	Create warehouse
Stock	GET	/warehouse-stock	stock.view	Stock balances
Movements	GET	/inventory-movements	movements.view	Movement history
Stock Takes	GET	/stock-takes	stock_takes.view	List stock takes
Stock Takes	POST	/stock-takes	stock_takes.manage	Create stock take
Stock Takes	POST	/stock-takes/{id}/post	stock_takes.manage	Post adjustments
Sales	GET	/sales-invoices	sales.view	List sales invoices
Sales	POST	/sales-invoices	sales.manage	Create sales invoice
Sales	PATCH	/sales-invoices/{id}	sales.manage	Update invoice
Sales	POST	/sales-invoices/{id}/confirm	sales.manage	Confirm (stock-out)
Sales	DELETE	/sales-invoices/{id}	sales.delete	Delete draft

Module	Method	Endpoint	Permission	Description
Purchases	GET	/purchase-invoices	purchases.view	List purchase invoices
Purchases	POST	/purchase-invoices	purchases.manage	Create purchase invoice
Purchases	POST	/purchase-invoices/{id}/confirm	purchases.manage	Confirm (stock-in + cost update)
Payments	GET	/payments	payments.view	List payments
Payments	POST	/payments	payments.manage	Create payment
Currencies	GET	/currencies	currencies.view	List currencies
Currencies	POST	/currencies	currencies.manage	Create currency
FX Rates	GET	/currency-rates	currency_rates.view	List exchange rates
FX Rates	POST	/currency-rates	currency_rates.manage	Create exchange rate
Accounts	GET	/accounting/accounts	accounting.view	Chart of accounts
Accounts	POST	/accounting/accounts	accounting.manage	Create account
Journals	GET	/accounting/journal-entries	accounting.view	List journal entries
Journals	POST	/accounting/journal-entries	accounting.manage	Create entry
Reports	GET	/accounting/reports/trial-balance	accounting.reports	Trial balance
Reports	GET	/accounting/reports/income-statement	accounting.reports	Income statement
Reports	GET	/accounting/reports/balance-sheet	accounting.reports	Balance sheet
HR	GET	/hr/departments	hr.view	List departments
HR	POST	/hr/departments	hr.manage	Create department
HR	GET	/hr/employees	hr.view	List employees
HR	POST	/hr/employees	hr.manage	Create employee
HR	POST	/hr/attendance	hr.manage	Record attendance
HR	POST	/hr/payroll/run	hr.payroll	Run payroll
HR	GET	/hr/leave-requests	hr.view	List leave requests
HR	POST	/hr/leave-requests	hr.manage	Create leave request
HR	POST	/hr/leave-requests/{id}/status	hr.manage	Approve/reject
Projects	GET	/projects/	projects.view	List projects
Projects	POST	/projects/	projects.manage	Create project
Projects	POST	/projects/{id}/costs	projects.manage	Add cost line
Projects	POST	/projects/{id}/complete	projects.manage	Mark complete
Mfg	GET	/manufacturing/boms	manufacturing.view	List BOMs
Mfg	POST	/manufacturing/boms	manufacturing.manage	Create BOM

Module	Method	Endpoint	Permission	Description
Mfg	GET	/manufacturing/work-orders	manufacturing.view	List work orders
Mfg	POST	/manufacturing/work-orders	manufacturing.manage	Create work order
Mfg	POST	/manufacturing/work-orders/{id}/finish	manufacturing.manage	Finish (stock-in)
POS	GET	/pos/sessions	pos.view	List POS sessions
POS	POST	/pos/sessions	pos.manage	Open session
POS	POST	/pos/sessions/{id}/close	pos.manage	Close session
POS	GET	/pos/orders	pos.view	List POS orders
POS	POST	/pos/orders	pos.manage	Create POS order
Reports	GET	/reports/sales-summary	accounting.reports	Sales summary
Reports	GET	/reports/stock-value	stock_takes.view	Stock value report
Reports	GET	/reports/low-stock	stock_takes.view	Low stock items
Reports	GET	/reports/project-costs	projects.view	Project cost report